

Project W: Functional formal verification of ZKsync's on-chain verifier

Julian Sutherland



Overview of the project

Goal: Mechanize a proof of functional correctness of the ZKsync on-chain SNARK verifier in the EasyCrypt proof assistant.

Overview of the project

Goal: Mechanize a proof of functional correctness of the ZKsync on-chain SNARK verifier in the EasyCrypt proof assistant.

Steps:

- ▶ Developing formal semantics of of the subset of Yul used in ZKsync's on-chain SNARK verifier, including the `expMod`, `ecAdd`, `ecMul` and `ecPairing` precompiles.
- ▶ Developing an extraction from Yul to EasyCrypt. (Thanks Igor Zhirkov)
- ▶ Compiling SNARK verifier to Yul and extracting into EasyCrypt.
- ▶ Functionally formally verifying the extraction using EasyCrypt's embedding of Probabilistic Relational Hoare Logic (PRHL).

EC modules and our Yul formalisation

```
type mem = (uint256, uint8) map.  
  
...  
  
(* uninterpreted functions *)  
op calldata : mem.  
op calldatasize: uint256.  
op callvalue : uint256.  
op keccak256_f : uint8 array -> uint256.  
  
...  
  
module Primops = {  
  var memory : mem  
  var ret_data : mem  
  var reverted : bool  
  
  proc mload(idx : uint256) : uint256 = {  
    return (PurePrimops.mload memory idx);  
  }  
  
  proc mstore(idx : uint256, val : uint256): unit = {  
    memory <- PurePrimops.mstore memory idx val;  
  }  
  
  proc mstore8(idx : uint256, val : uint256) : unit = {  
    memory <- PurePrimops.mstore8 memory idx val;  
  }  
  
  ...  
}
```

PRHL and Functional Verification

- ▶ PRHL statements:

$$\vdash \{P\} C_1 \sim C_2 \{Q\}$$

- ▶ P and Q are assertions relating the state of the two programs being related.
- ▶ The semantics are that, if these two programs are executed in two states related by P , then the final states will be related by Q .
- ▶ We use this to relate the Yul implementation to high-level algebraic programs.

Proof steps

- ▶ Implementation $\sim$ Low: Eliminating intermediate variables introduced by the compiler.
- ▶ Low $\sim$ Mid: Lifting a program writing in Yul, using memory and `UInt256`s and bytes as data representation to a program over arbitrary precision integers. This eliminates the possibility of any bugs due to address clashes and overflow in the representation.
- ▶ Mid $\sim$ High: Relating these programs over arbitrary precision integers with the algebraic objects used in the PLONK commitment specification.
- ▶ Mid $\sim$ High for verify function: Includes “deoptimisation”.

Example (pointNegate)

```
proc usr_pointNegate(usr_point : uint256): unit = {
  var _1, _2, usr_pY, tmp88, tmp89, _3, tmp90, _4, _5, tmp91, _6, _7;
  _1 <- (W256.of_int 32);
  _2 <- (usr_point + _1);
  tmp88 <@ Primops.mload(_2);
  usr_pY <- tmp88;
  tmp89 <- usr_pY;
  if ((tmp89 = (W256.of_int 0)))
  {
    tmp90 <@ Primops.mload(usr_point);
    _3 <- tmp90;
    if ((bool_of_uint256 _3))
    {
      _4 <- (W256.of_int STRING (*pointNegate: invalid point*));
      _5 <- (W256.of_int 26);
      tmp91 <@ usr_revertWithMessage(_5, _4);
    }
  }
  else {
    _6 <- (W256.of_int 21888242871839275222246405745257275088696311157297823662689037894645226208583);
    _7 <- (_6 - usr_pY);
    Primops.mstore(_2, _7);
  }
}
```

Example (pointNegate)

```
proc low(usr_point : uint256): unit = {
  var _2, usr_pY, tmp90, tmp91, _6, _7 : uint256;
  _2 <- (usr_point + W256.of_int 32);
  usr_pY <@ Primops.mload(_2);
  if (usr_pY = W256.zero) {
    tmp90 <@ Primops.mload(usr_point);
    if (bool_of_uint256 tmp90) {
      RevertWithMessage.low((W256.of_int 26), W256.of_int STRING);
    }
  }
  else {
    Primops.mstore(_2, (Q_MOD - usr_pY));
  }
}
```

Example (pointNegate)

```
proc low(usr_point : uint256): unit = {
  var _2, usr_pY, tmp90, tmp91, _6, _7 : uint256;
  _2 <- (usr_point + W256.of_int 32);
  usr_pY <@ Primops.mload(_2);
  if (usr_pY = W256.zero) {
    tmp90 <@ Primops.mload(usr_point);
    if (bool_of_uint256 tmp90) {
      RevertWithMessage.low((W256.of_int 26), W256.of_int STRING);
    }
  }
  else {
    Primops.mstore(_2, (Q_MOD - usr_pY));
  }
}

lemma pointNegate_extracted_equiv_low :
  equiv [
    Verifier_1261.usr_pointNegate ~ PointNegate.low :
      ={arg, glob PointNegate} ==>
      ={res, glob PointNegate}
  ].
proof.
...
qed.
```

Example (pointNegate)

```
proc low(usr_point : uint256): unit = {  
  var _2, usr_pY, tmp90, tmp91, _6, _7 : uint256;  
  _2 <- (usr_point + W256.of_int 32);  
  usr_pY <@ Primops.mload(_2);  
  if (usr_pY = W256.zero) {  
    tmp90 <@ Primops.mload(usr_point);  
    if (bool_of_uint256 tmp90) {  
      RevertWithMessage.low((W256.of_int 26), W256.of_int STRING);  
    }  
  }  
  else {  
    Primops.mstore(_2, (Q_MOD - usr_pY));  
  }  
}
```



```
proc mid(point: int*int) : (int * int) option = {  
  var ret;  
  if (point.`1 <> 0 /\ point.`2 = 0) {  
    ret <- None;  
  } else {  
    ret <- Some (point.`1, (-point.`2) %% Constants.Q);  
  }  
  return ret;  
}
```

Example (pointNegate)

```
proc mid(point: int*int) : (int * int) option = {  
  var ret;  
  if (point.`1 <> 0 /\ point.`2 = 0) {  
    ret <- None;  
  } else {  
    ret <- Some (point.`1, (-point.`2) %% Constants.Q);  
  }  
  return ret;  
}
```



```
proc high(p: g): g = {  
  return G.inv p;  
}
```

Final high-level specification

Future Work

- ▶ Convert to equivalent interactive sigma protocol and prove zk protocol safety properties (See Denis Firsov's work):
 - ▶ Soundness.
 - ▶ Proof of Knowledge.
 - ▶ Cannot do Completeness and Zero-knowledge properties without formalisation of the proof generator.
- ▶ Embedding of PRHL into Lean.
- ▶ Formalisation of safety properties of the PLONK and PLOOKUP protocols.
- ▶ Mechanise proof of correctness of Fiat-Shamir heuristic.
- ▶ Formalisation of other sigma protocols.

Thank you for listening! Any questions?



NETHERMIND

